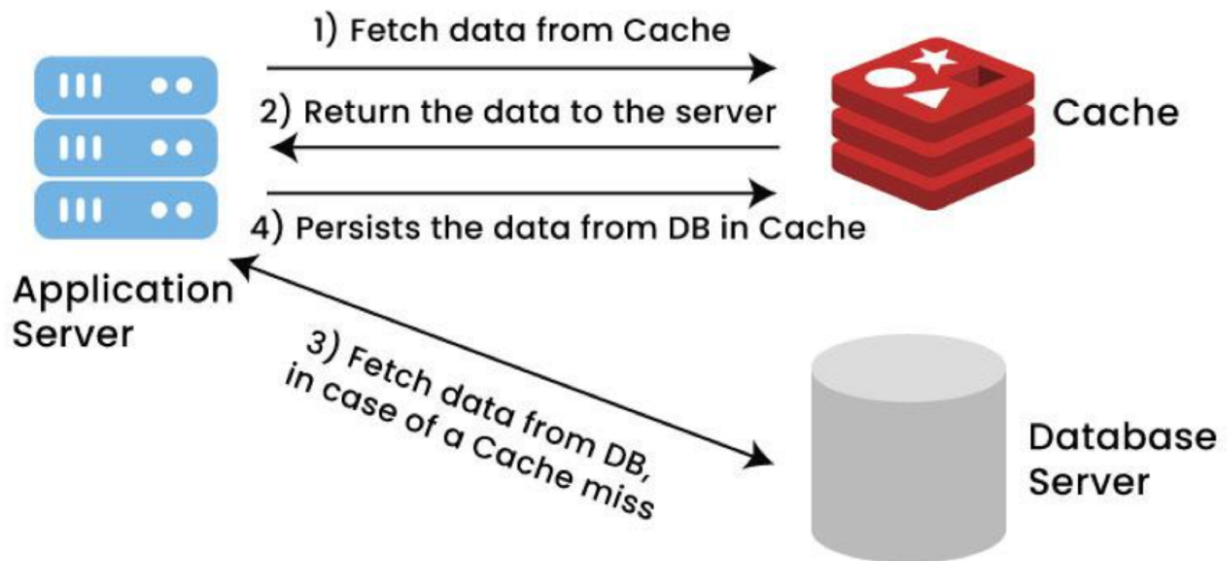


Caching System Design Concept

DAY 158
15 / Dec / 2025

Caching is a concept that involves storing frequently accessed data in a location that is easily and quickly accessible. The purpose of caching is to improve the performance and efficiency of a system by reducing the amount of time it takes to access frequently accessed data.

- Caching acts as the local store for the data and retrieving the data from this local or temporary storage is easier and faster than retrieving it from the database.
- In a typical web application, we can add an application server cache and an in-memory store like [Redis](#) alongside our application server.



Example: In twitter, when a tweet become viral a huge number of clients requests the same tweet, so to reduce the number of calls to the database, we can use cache and the tweets can be provided much faster.

How Does Cache Work?

Web application stores data in a database. Reading data from the database needs network calls and I/O operations which is a time-consuming process. Cache reduces the network calls to the database and speeds up the performance of the system.

- When the first time a request is made a call will have to be made to the database to process the query. This is known as a cache miss.
- Before giving back the result to the user, the result will be saved in the cache.
- When the second time a user makes the same request, the application will check your cache first to see if the result for that request is cached or not.
- If it is then the result will be returned from the cache. This is known as a cache hit.
- The response time for the second time request will be a lot less than the first time.

Why not store all data in cache?

As you know there are many benefits of the cache but that doesn't mean we will store all the information in the cache memory for faster access, we can't do this for multiple reasons, such as:

- Hardware of the cache which is much more expensive than a normal database.

- Also, the search time will increase if you store tons of data in your cache.
- Cache is typically a volatile storage, meaning data is lost if the system crashes or restarts. For critical and long-term data, storing it only in cache would risk data loss.
- In short, a cache needs to have the most relevant information according to the request which is going to come in the future.

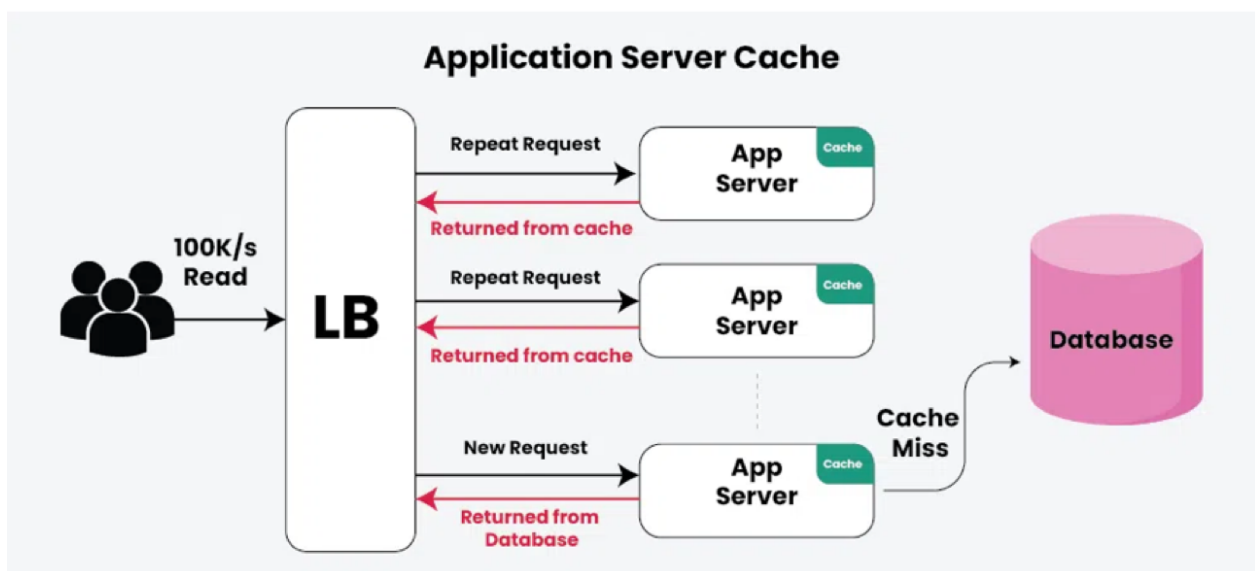
Types of Cache

In common there are four types of Cache:

1. Application Server Cache

An Application Server Cache is a storage layer within an application server that temporarily holds frequently accessed data, so it can be quickly retrieved without needing to go back to the main database each time. This helps applications run faster by reducing the load on the database and speeding up response times for users.

Example: When an app frequently needs certain data, the application server can store this data in the cache. When users request it, app can instantly provide the cached version instead of processing a full database query.



Drawbacks of Application Server Cache:

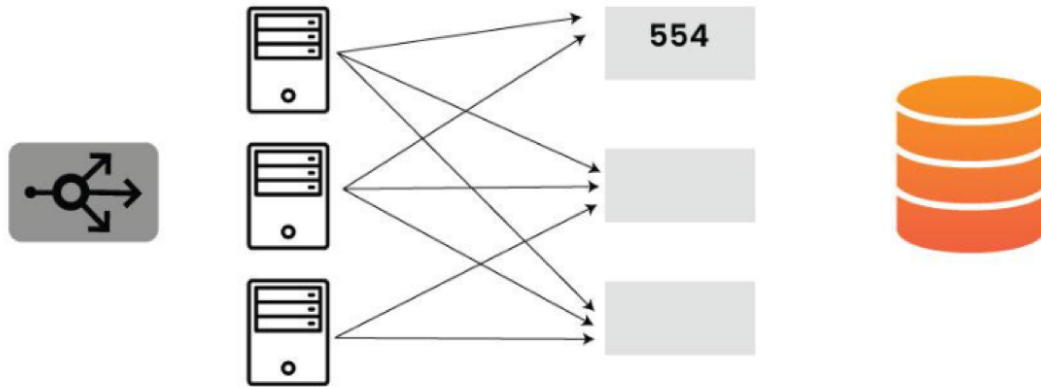
- When you add multiple servers to handle a high volume of requests.
- With several servers, a load balancer sends requests to different nodes, but each node only has its own cache and doesn't know about the cached data on other nodes.
- This results in many cache misses, meaning the data has to be re-fetched frequently, slowing things down.

Note: To fix this, there are two main options: Distributed Cache and Global Cache.

2. Distributed Cache

In the distributed cache, each node will have a part of the whole cache space and then using the consistent hashing function each request can be routed to where the cache request could be found.

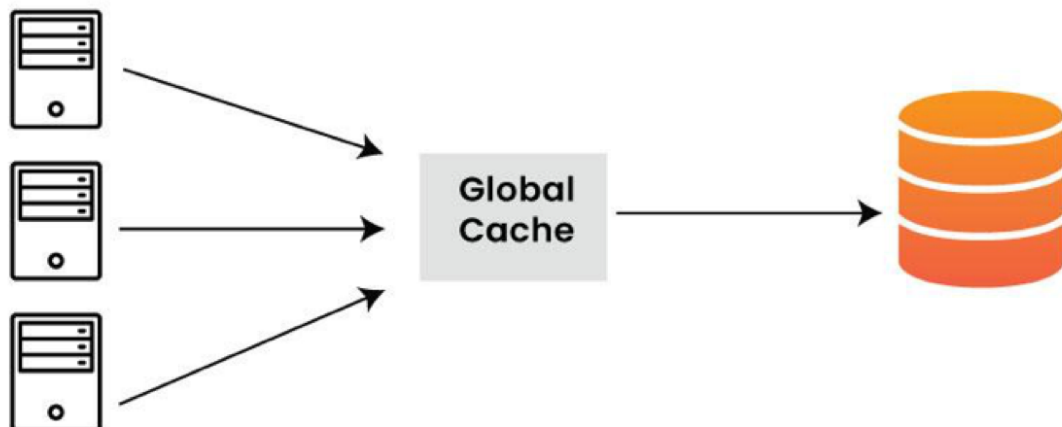
- Each of its nodes will have a small part of the cached data.
- To identify which node has which request the cache is divided up using a consistent hashing function, so that each request can be routed to where the cached request could be found.
- If a requesting node is looking for a certain piece of data, it can quickly know where to look within the distributed cache to check if the data is available.



3. Global Cache

As the name suggests, you will have a single cache space and all the nodes use this single space. Every request will go to this single cache space. There are two kinds of the global cache

- First, when a cache request is not found in the global cache, it's the responsibility of the cache to find out the missing piece of data from anywhere underlying the store (database, disk, etc).
- Second, if the request comes and the cache doesn't find the data then the requesting node will directly communicate with the DB or the server to fetch the requested data.



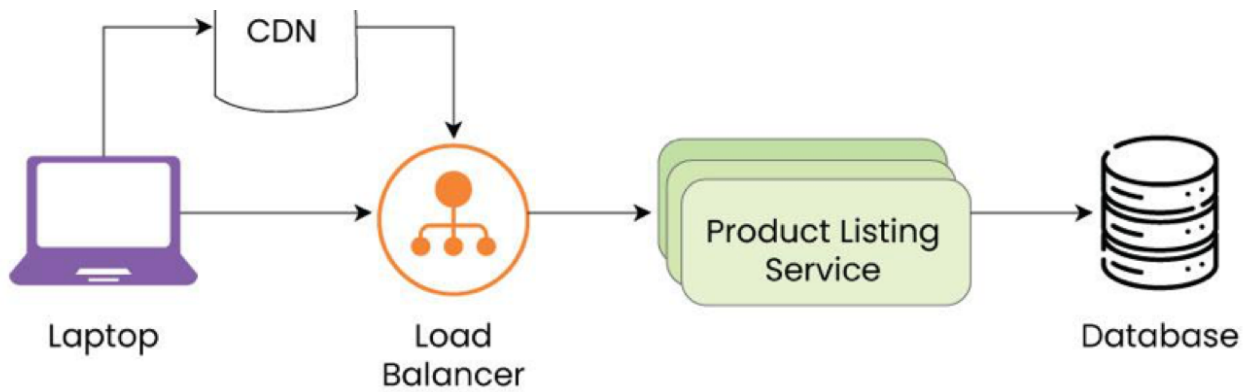
4. CDN (Content Delivery Network)

A [CDN](#) is essentially a group of servers that are strategically placed across the globe with the purpose of accelerating the delivery of web content. A CDN:

- Manages servers that are geographically distributed over different locations.
- Stores the web content in its servers.
- Attempts to direct each user to a server that is part of the CDN and close to the user so as to deliver content quickly.
- Used where a large amount of static content is served by the website.

Note: It can be an HTML, CSS, JavaScript Files, pictures, videos, etc. First, request ask CDN for data, if it exists then the data will be returned. If not, CDN will query the backend servers and then cache it locally.





Applications of Caching

Caching is used in many areas to speed up processes, reduce load and make systems more efficient. Below are some common applications of caching:

- **Web Page Caching:** In order to speed up loading times in the future, browsers save copies of frequently visited websites. This saves bandwidth and shortens the time it takes for a web page to load.
- **Database Caching:** Frequent database queries can strain servers and cause lag. Caching allows apps to quickly retrieve frequently used data without repeatedly asking the database by storing it in memory.
- **Content Delivery Networks (CDNs):** CDNs use caching to keep copies of data (such as pictures and videos) in several places throughout the globe. This enhances website performance by enabling visitors to obtain content more quickly from a nearby server.
- **Session Caching:** Applications store session data in a cache to remember user information (like login status) between visits, making the experience seamless and personalized without needing to re-login.
- **API Response Caching:** Frequently requested API data, like stock prices or weather data, can be cached so responses are faster, reducing the load on the server and delivering data in real-time.

Cache Invalidation Strategies

For systems that use caching to improve performance, cache invalidation is essential. Data is temporarily kept for faster access when it is cached. However, the cached version goes out of date if the original data changes.

- In order to guarantee that users obtain the most recent information, [cache invalidation techniques](#) make sure that out-of-date records are either updated or deleted.
- Common strategies include time-based expiration, where cached data is discarded after a certain time and event-driven invalidation, triggered by changes to the underlying data.
- Proper cache invalidation optimizes performance and avoids serving users with obsolete or inaccurate content from the cache.

Eviction Policies of Caching

For caching systems to effectively manage their limited cache capacity, eviction policies are essential. An [eviction policy](#) decides which existing item to remove when the cache is full and a new item needs to be stored.

- The Least Recently Used (LRU) policy is a popular strategy that eliminates the item that has been accessed the least recently. According to this assumption, items which have been used recently are more likely to be utilized again shortly.
- Another method is the Least Frequently Used (LFU) policy, removing the least frequently accessed items.
- Alternatively, there's the First-In-First-Out (FIFO) policy, evicting the oldest cached item.

Pros of Caching

As it maximizes resource utilization, reduces server loads and enhances overall scalability, caching is a helpful technique in software development.

- **Improved performance:** By significantly reducing down on the time it takes to get frequently used data, caching can enhance system responsiveness and performance.
- **Reduced load on the original source:** By significantly reducing down on the time it takes to get frequently used data

- **Reduced load on the original source:** By significantly reducing down on the time it takes to get frequently used data, caching can enhance system responsiveness and performance.
- **Cost savings:** Caching can reduce the need for expensive hardware or infrastructure upgrades by improving the efficiency of existing resources.

Cons of Caching

Despite its advantages, caching comes with drawbacks also and some of them are:

- **Data inconsistency:** If cache consistency is not maintained properly, caching can introduce issues with data consistency.
- **Cache eviction issues:** If cache eviction policies are not designed properly, caching can result in performance issues or data loss.
- **Additional complexity:** Caching can add additional complexity to a system, which can make it more difficult to design, implement and maintain.